Chapter 2

Logic minimization

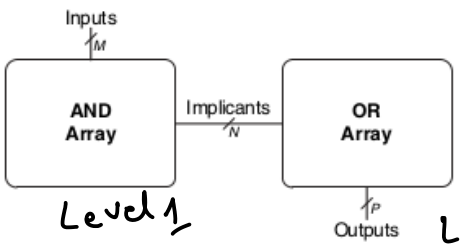
2.1 Logic minimization

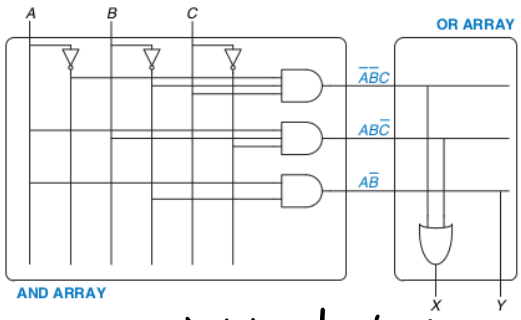
A general optimization criteria for multi-level logic are to Minimize some combination of:

1. Area occupied by the logic gates and interconnect;
2. the Critical Path Delay of the longest path through the logic;
3. the Degree of Testability of the circuit, measured in terms of the percentage of faults covered by a specified set of test vectors, for an appropriate fault model (Eg., single stuck faults, multiple stuck faults, etc.);
4. Power consumed by the logic gates.

In this course, we will start with two-level multi-input circuits and a criteria based on the number of gates/transistors/diodes.

2.2 Programmable Logic Arrays





2.3 Two-level circuits

The cost that we are going to consider in this class depend upon:

1. Number of gates.
2. Number of input to the gates.

$X = \bar{A}\bar{B}C + AB\bar{C} + A\bar{B}$

we don't count NOT gates, only AND and OR gates

cost $(\bar{A}\bar{B}C)$ = 1 AND + 3 inputs = 4

cost $(AB\bar{C})$ = " = 4

cost $(A\bar{B})$ = 1 AND + 2 inputs = 3

cost $(\bar{A}\bar{B}C + AB\bar{C} + A\bar{B})$ = 2 OR + 3 inputs = 4

Total cost $\propto$ 15 ← Use this to compare two Boolean expressions / circuits for simplicity

More gates need more transistors, more area on the chip. More-inputs the gate need more transistors within each gate. Number of gate inputs can be considered secondary criterion to the number of gates.

Example 2.1. *Find the cost of the following Boolean expression* $X = \bar{A}\bar{B}C + AB\bar{C} + A\bar{B}$.

Problem 2.1 (5 marks)**.** *Find the cost of the following Boolean expression* $X = A\bar{B}C + \bar{A}B\bar{C} + \bar{B}C$.

2.4 Terminology for K-maps

Running Example: $f = \sum m(0,1,2,3,7) = \bar{x}_1 + x_1x_2x_3$

Literal A single variable or its complement. Example: $\bar{x}_1, x_1, x_2, x_3$

Implicant A product term which is true for a function. All minterms are implicants. Example: $x_1x_2x_3$, $\bar{x}_1$, $m_0 = \bar{x}_1\bar{x}_2\bar{x}_3$, $\bar{x}_1x_3$, $\bar{x}_1\bar{x}_3$.

Prime Implicant An implicant that cannot be combined into fewer literals. Example: $\bar{x}_1$, x_2x_3.

Essential Prime Implicant An implicant that cannot be combined into fewer literals. Example: x_2x_3.

EPI: is a PI that "covers" a 1 not "covered" by any other PI.

Cover : List of Prime Implicants that account for all $f = 1$.

Cost : Number of gates (excluding not gate on literals) and number of inputs to each gate.

Example 2.2. *Find minimum cost expression for the function* $f(x_1, x_2, x_3) = \prod M(4,5,6)$

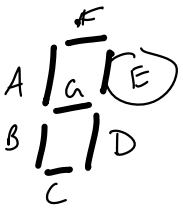
1. Find the biggest and fewest "groupings" of 1s in the K-map

1. Find all the prime implicants (biggest groupings)
2. Fewest: Note down all the essential prime implicant (EPIs).
   Remove the 1s covered by EPIs.
3. Repeat 2 until all 1s are covered.
4. Finding the minimum cover for PI algorithms (Quine-McCluskey algo).

Problem 2.2 (10 marks)**.** *Find minimum cost expression for the function* $f(x_1, x_2, x_3) = \prod M(2,5,6)$

2.4.1 Incompletely specified functions or Don't cares

BCD Value				LED Segment
D_3	D_2	D_1	D_0	E
0	0	0	0	0
0	0	0	1	1
0	0	1	0	0
0	0	1	1	1
0	1	0	0	1
0	1	0	1	1
0	1	1	0	0
0	1	1	1	1
1	0	0	0	0
1	0	0	1	1
1	0	1	0	d
1	0	1	1	d
1	1	0	0	d
1	1	0	1	d
1	1	1	0	d
1	1	1	1	d



0-9

don't care

Treat don't cares as 0/1 depending on which gives you a simpler circuit.

Example 2.3. *Find minimum cost expression for the function*

$$f(x_1, \ldots, x_4) = \sum m(2, 4, 5, 6, 10) + D(12, 13, 14, 15)$$

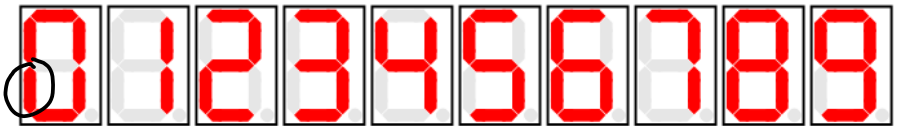
Problem 2.3 (10 marks)**.** *Find minimum cost expression for the function*

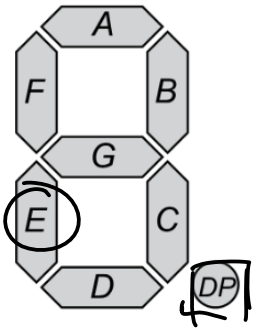
$$f(x_1, \ldots, x_4) = \sum m(0, 2, 4, 6, 7, 8, 9, 13) + D(1, 12, 15)$$

2.5 A few more Boolean problems

Example 2.4. *Simplify the following Boolean expression:*

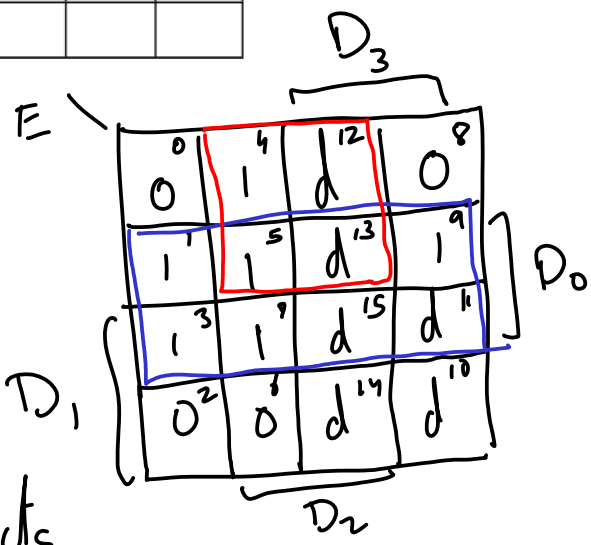
$$f = x_1\bar{x}_3\bar{x}_4 + x_2\bar{x}_3\bar{x}_4 + x_1\bar{x}_2\bar{x}_3$$





	BCD Value				LED Segment						
	D_3	D_2	D_1	D_0	G	F	E	D	C	B	A
0	0	0	0	0			0				
1	0	0	0	1			1				
2	0	0	1	0			0				
3	0	0	1	1			1				
4	0	1	0	0			1				
5	0	1	0	1			1				
6	0	1	1	0			0				
7	0	1	1	1			1				
8	1	0	0	0			0				
9	1	0	0	1			1				

1	0	1	0	d
1	0	1	1	d
1	1	0	0	d
1	1	0	1	d
1	1	1	0	d
1	1	1	1	d



Ans $E = D_0 + D_2\bar{D}_1$ sum of products

1 AND +2 inputs cost α 6

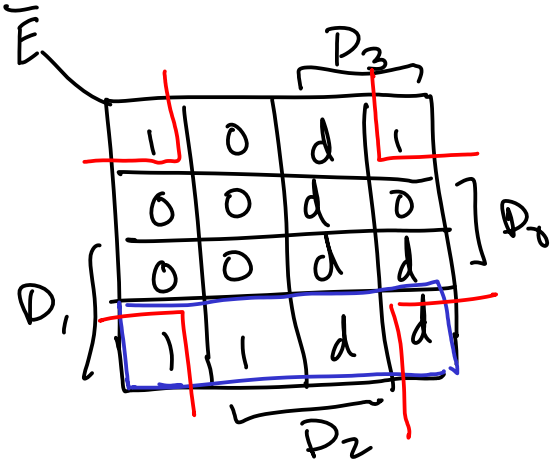
$\bar{E} = D_1\bar{D}_0 + \bar{D}_2\bar{D}_0$

$E = (\bar{D}_1 + D_0)(D_2 + D_0)$ product of sums

3 3 3

cost α 9

1 OR +2 inputs 3



You don't have to check both sums of products and product of sums unless explicitly asked in the question.

Problem 2.4 (20 marks). *A simple security system for two doors consists of a card reader and a keypad.*

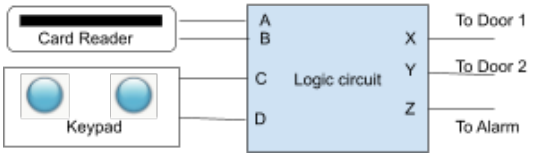
A person may open a particular door if he or she has a card containing the corresponding code and enters an authorized keypad code for that card. Note that card-code and keypad-code are different. The outputs from the card reader are given in the table below.

To unlock a door, a person must hold down the proper keys on the keypad and, then, insert the card in the reader. The authorized keypad code for door 1 is 10, and the authorized keypad code for door 2 is 11. If the card has an invalid code or if the wrong keypad code is entered, the alarm will ring when the card is inserted. If the correct keypad code is entered, the corresponding door will be unlocked when the card is inserted.

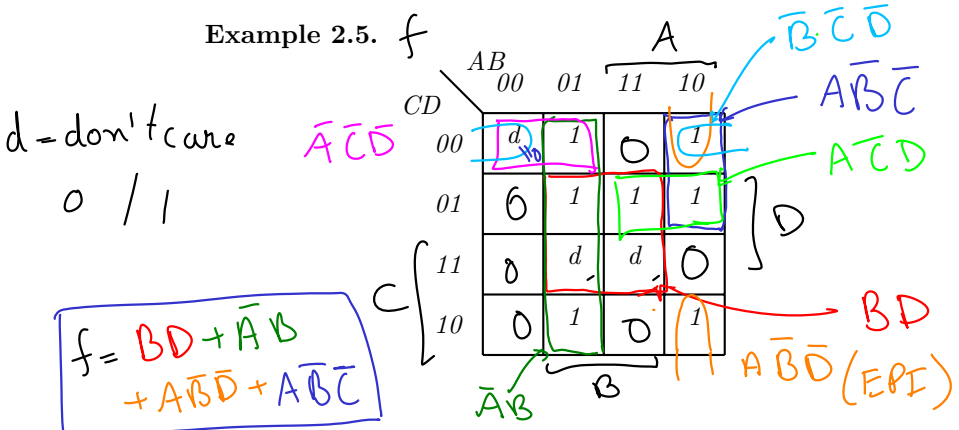
Design the logic circuit for this simple security system. Your circuit's inputs will consist of a card code AB, and a keypad code CD. The circuit will have three outputs XYZ (if X is 1, door 1 will be opened; if Y is 1, door 2 will be opened; if Z 1, the alarm will sound).

Find the minimal cost two-level circuit using K-maps for X, Y, Z. Provide the minimal cost. (It can be either of SOP/POS forms)

	A	B
No card inserted	0	0
Valid card-code for door 1	0	1
Valid card-code for door 2	1	1
Invalid card code	1	0



Example 2.5.



Problem 2.5 (10 marks). *Find the minimum SOP (sum of products) and POS (product of sum) expression for the function* $f(a, b, c, d) = \prod M(5, 7, 13, 14, 15) \cdot \prod D(1, 2, 3, 9)$

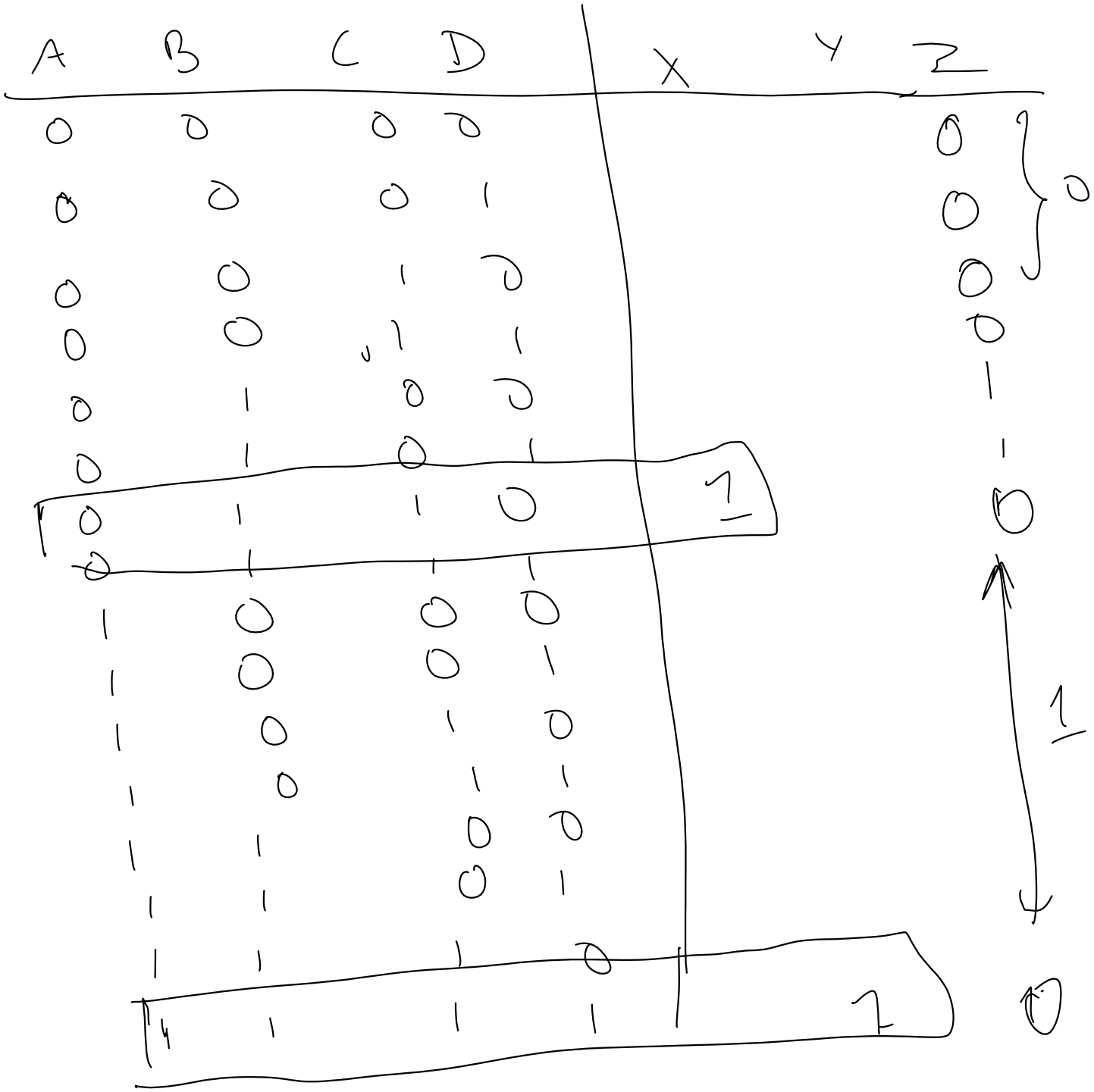
Problem 2.6 (10 marks). *If the Sum of Products (SOP) form for* $\bar{f} = AB\bar{C} + \bar{A}\bar{B}$*, then give the Product of Sums (POS) form for* f*.*

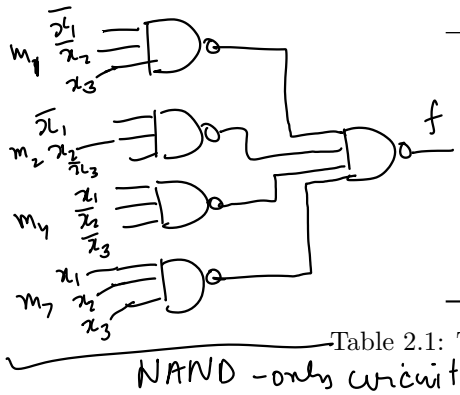
Problem 2.7 (10 marks). *Use DeMorgan's Theorem to find* f *if* $\bar{f} = (A + \bar{B}C)D + EF$*.*

Example 2.6. *Implement the function in Table 2.1 using only NAND gates.*

Problem 2.8 (10 marks). *Implement the function in Table 2.1 using only NOR gates.*

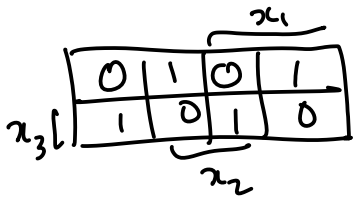
Problem 2.9 (10 marks). *Find the minimum-cost Sum of Products (SOP) and Product of Sums (POS) forms for the function* $f(x_1, x_2, x_3) = m(1, 3, 4, 5)$*. Chose the minimum-cost expression by comparing Product of Sums (POS) and Sum of Products (SOP) forms.*


A
B
C
D
X
Y
Z
0
1
1
1
0



Row	x_1	x_2	x_3	f
0	0	0	0	0
1	0	0	1	1
2	0	1	0	1
3	0	1	1	0
4	1	0	0	1
5	1	0	1	0
6	1	1	0	0
7	1	1	1	1

Table 2.1: Truth table for a 3-way light switch



$f = m_1 + m_2 + m_4 + m_7$

Problem 2.10 (10 marks)**.** *Find the minimum-cost Sum of Products (SOP) and Product of Sums (POS) forms for the function* $f(x_1, x_2, x_3) = \sum m(1, 5, 7) + D(2, 4)$.

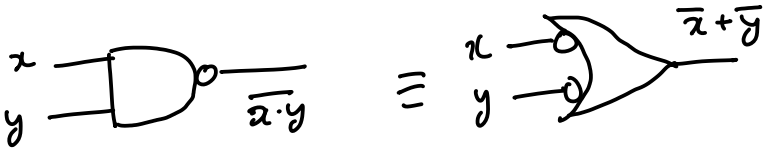
Problem 2.11 (10 marks)**.** *Find the minimum-cost Sum of Products (SOP) and Product of Sums (POS) forms for the function* $f(x_1, x_2, x_3, x_4) = \prod M(1, 2, 4, 5, 7, 8, 9, 10, 12, 14, 15)$. *Chose the minimum-cost expression by comparing Product of Sums (POS) and Sum of Products (SOP) forms.*

Problem 2.12 (10 marks)**.** *Derive a minimum-cost realization of the four-variable function that is equal to 1 if exactly two or exactly three of its variables are equal to 1; otherwise it is equal to 0.*

How to design two-level circuits using only NAND gates?

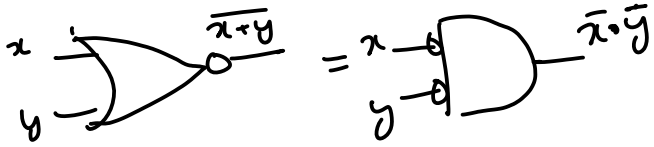
De Morgan Theorem in ANSI symbols

$\overline{x \cdot y} = \bar{x} + \bar{y}$



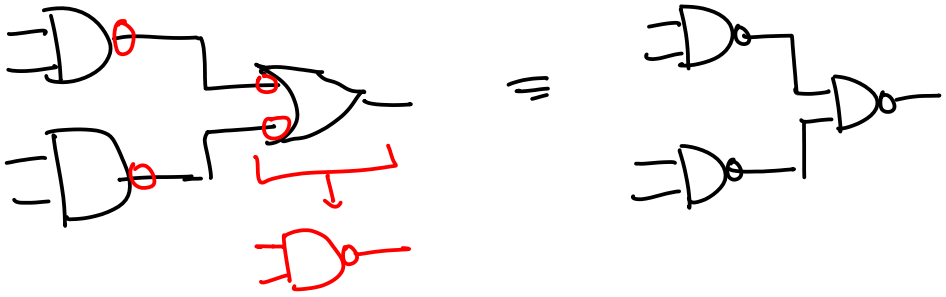
$\overline{x+y} = \bar{x} \cdot \bar{y}$

Bubble pushing



When you push a bubble through a gate,
it converts OR to an AND;
converts AND to an OR;

Sum of Products ⟵ NAND only circuit



Product of sums circuit ⟵ NOR only circuit

```
1  // this should match the top level module name you set
2  module yourprojecttop(
3      input [3:0] SW, // make sure pin assignments are set
4      output [6:0] HEX0 // make sure pin assignments are set
5  );
6      wire [3:0] BCDValue; // wire is same as logic in System V
7      wire [6:0] LED_Segment;
8
9      // Renaming the wires into something more
10     // representative of the values at the switches
11     assign BCDValue[3:0] = SW[3:0]; // continuous assignment
12     // Renaming the output as well
13     // Note that the order does not matter.
14     assign HEX0[6:0] = LED_Segment[6:0];
15
16     // Renaming again for convenience of typing and
17     // readability
18     wire d3, d2, d1, d0; // Declares single bit wires
19     // Curly braces {} break bits into into single
20     // bit variables. They can also be used to
21     // concatenate (join) bits on the right hand side.
22     assign {d3, d2, d1, d0} = BCDValue[3:0];
23
24     assign LED_Segment[0] = 0; // your eqn for segment A;
25     assign LED_Segment[1] = 0; // your eqn for segment B;
26     assign LED_Segment[2] = 0; // your eqn for segment C;
27     assign LED_Segment[3] = 0; // your eqn for segment D;
28     assign LED_Segment[4] = 0; // your eqn for segment E;
29     assign LED_Segment[5] = 0; // your eqn for segment F;
30     assign LED_Segment[6] = 0; // your eqn for segment G;
31 endmodule
```

Verilog → wire [3:0] BCDValue;

SV logic ← multiple meanings (reg = Verilog)

HEX0 [6:0] ← LSB (Right most) ← 7 bits

{d3, d2, d1, d0} = BCDValue[3:0];

=SW[0] | SW[2] & (~SW[1]);

Structural Verilog

Behavioral Verilog

```
----------

module BCD_DisplayBehav(
    input [3:0] BCDValue,
    output [6:0] LED_Segment
);
// All behavioral verilog goes in always block.
// Instead of curly braces, Verlog uses begin-end
// blocks. Behavioral verilog syntax can only be
// used inside some kind of always_* blocks.
always_comb begin
  // (1) output 0 indicates a lit segment.
  // (2) 7'b means 7 bit binary number.
  // (3) _ underscore is only used to separate
  //     large numbers.
  // (4) 4'd means the following it is a 4 bit decimal.
  // (5) <= means non-blocking  assignment. The next
  //     statement will execute without waiting for
  //     the previous one to execute. This is
  //     opposed to =, which is blocking assignment.
  //     You cannot use "assign" keyword in
  //     behavioral verilog.
  // (6) case (varable) matches number before
  //     colon: if the matches is true the statement or
  //     begin-end block after the colon:.
  // (7) if the case (variable) does not match anything
  //     then the circut must behave what the default
  //      blocks says.
    case (BCDValue)
        //                    gfe_dbca
        4'd0 : LED_Segment <= 7'b100_0000;  //
        // if BCDValue matches 4 bit decimal 1,
        // then non-blockingly assign LED_Segment to 7 bit binary 111_100
        4'd1 : LED_Segment <= 7'b111_1001;    // ---0---
        4'd2 : LED_Segment <= 7'b010_0100;    // |     |
        4'd3 : LED_Segment <= 7'b011_0000;    // 5     1
        4'd4 : LED_Segment <= 7'b001_1001;    // |     |
        4'd5 : LED_Segment <= 7'b001_0010;    // ---6---
        4'd6 : LED_Segment <= 7'b000_0010;    // |     |
        4'd7 : LED_Segment <= 7'b111_1000;    // 4     2
        4'd8 : LED_Segment <= 7'b000_0000;    // |     |
        4'd9 : LED_Segment <= 7'b001_0000;    // ---3---
        // Verilog bits can take four values 1, 0, x and z.
        // z stands for "floating" or "disconnected" wire
        // x stands for "unknown" or "don't care"
        default : LED_Segment <= 7'bxxx_xxxx;
```

non-blocking assignment

blocking assignment

floating

1, 0, z, x

↑ unknown

AND

↓ \ →	0	1	z	x
0	0	0	z	x
1	0	1	1	1
z	z	1	z	z
x	x	1	z	x